

PIPELINE REPORT

How DeciWaves works

DeciWaves

DeciWaves turns your own installed copies of three games (Death Stranding, Horizon Zero Dawn Remastered, Horizon Forbidden West) into a spreadsheet of every spoken line and a set of story-ordered MP3 files. You point it at a game you own; it reads, and never changes, the game's files, works out which recorded clip carries which subtitled line and who said it, puts those lines in story order, and stitches them into MP3 reels of about 290 MB each with a tracklist beside every one. There is now a desktop GUI, and it is the recommended way to run everything; underneath it drives the same command-line tool. Windows only. It ships no game content of its own.

Covers all three games. They share one skeleton but the middle differs per game, so the report follows the shared five stages and notes where each game diverges. It also covers the desktop GUI added since the last report. This supersedes the report dated 17 July 2026: two fragilities that one listed have since been fixed, one claim it made is now out of date, and a whole GUI has landed. What follows describes the pipeline as it stands on 22 July 2026. Where this and the code disagree, the code is right.

Generated 22 July 2026.

Read the code, not just this. Where the two disagree, the code is right.

1.

What this is

DeciWaves is a tool for someone who owns these three games and wants the voice acting as listenable audio, indexed and in story order. The unit of work is one game. For each one, “done” means two things: a manifest, a CSV spreadsheet with one row per voice line carrying a stable id, the line’s internal name, its speaker where that can be derived, its scene and category, its language, and a path to a playable clip; and a set of MP3 reels assembled from those clips in narrative order.

There are two ways to drive it. The desktop GUI is the front the README now leads with, and the command-line tool is the same engine underneath. The GUI runs by launching `deciwaves` commands, so the stages below describe both at once.

Three rules shape everything. It is read-only against any install it touches, so it never repacks or edits game files. It ships code, never game content, so copyrighted script text is something you supply yourself and point the tool at. And it is Windows only, because it shells out to Windows decoder binaries for the audio.

The three games look similar from the outside and are genuinely different underneath. Death Stranding stores dialogue in Decima engine resources that name each line’s audio directly. The two Horizon games use newer, different archive formats that do not, so most of the engineering lives in working out which recorded clip goes with which line. Treat every format detail here as reverse engineered and likely, not certified: it comes from inspecting files, not from documentation.

2.

At a glance

1

Configure and check

Point it at a game you own; fetch the decoder tools; confirm what is present.



2

Catalog

Read the install and list every dialogue line: id, name, speaker, subtitle, scene.



3

Bind

Attach the actual recorded clip to each listed line. The hard part; each game solves it differently.



4

Order

Arrange the bound lines in story sequence.



5

Render

Decode the clips and pack them into MP3 reels, each with a tracklist.

The five stages are conceptual. Death Stranding skips Bind entirely, because its catalog already names each line's audio. The Horizon games fold Order into their manifest rather than running it as a separate step. The command that runs a game end to end (`deciwaves <game> run`) executes only the stages that game actually has, and skips any whose work is already done. The GUI shows these same stages as a strip of chips and runs them through that same command.

3.

Stage by stage

Drive it: the desktop GUI or the CLI

The GUI is a thin front-end. It does not re-implement the pipeline: every action runs the same `deciwaves` command a terminal user would type, as a separate process, with its output streamed live into a log panel. The stage strip is built from the same list the `run` command uses, so the buttons cannot drift from what the tool does. Running `deciwaves` with no arguments opens the GUI when the extra is installed, and falls back to a guided text prompt otherwise.

Two rules the GUI enforces are worth knowing. Only one pipeline job runs at a time, app-wide: Scan, Process, a re-run, and an Export render cannot overlap. And before any GPU stage it pops a confirmation if no graphics card is visible, because that stage on a CPU can take days. Cancelling a job is safe, because the tool resumes from where it stopped.

Only two things run inside the GUI rather than as a `deciwaves` job: playing a single line as audio, and dumping the ticked lines to WAV files. Both reuse the render stage's decoders and clip cache, so work is never done twice. A first-run "guide rail" across the top points at the next thing to do; clicking it only moves you to the right control, and never launches a job itself.

IN	your clicks, or your typed commands
DOES	turns either into <code>deciwaves</code> runs and shows their progress
OUT	the same files the CLI produces, in the same workspace
NEEDS	the <code>gui</code> extra (PySide6) for the desktop app; nothing extra for the CLI
FILES	<code>gui/shell.py</code> <code>gui/job_controller.py</code> <code>gui/cli_command.py</code> <code>cli/main.py</code>

Configure and check readiness

`deciwaves setup` fetches `ffmpeg`, `vgmstream-cli` and `VGAudioCli` into `%LOCALAPPDATA%\DeciWaves\tools` and records where each game and the Oodle DLL live. It merges rather than overwrites: an omitted flag keeps the value you saved before, and passing an empty string clears one, so registering a second game never blanks the first game's paths. A corrupt or hand-edited `config.json` is ignored with a warning and the run starts from an empty config, rather than crashing every command.

`deciwaves doctor` prints a readiness list: each decoder tool present or missing with the exact fix, each game found or not configured, and whether the `asr` extra and a CUDA GPU are visible. A game you do not own reports “not configured” and never counts as a failure. The GUI wraps both commands unchanged (`doctor` in a machine-readable `--json` mode) and adds only display sense on top: it treats a missing GPU as a real blocker for the Horizon games but as information only for Death Stranding, and reconciles the Setup and Doctor panels so they cannot contradict each other.

IN	an internet connection; the game or games you own
DOES	downloads the decoder tools and records where things live
OUT	a saved <code>config.json</code> plus the tools on disk
NEEDS	<code>github.com</code> reachable; for the Horizon games, a CUDA GPU and the <code>asr</code> extra
FILES	<code>cli/setup.py</code> <code>cli/doctor.py</code> <code>gui/setup_model.py</code> <code>gui/doctor_model.py</code>

Catalog: list every line

Death Stranding reads the Decima sentence resources directly and records each line's id, internal name, category, scene, speaker and English subtitle, alongside the path to its audio. Horizon Zero Dawn cannot do this: its archive exposes only path hashes, not names, so the catalog recovers sentence-core paths by scanning file bodies for readable path strings, then reads the same fields. Forbidden West takes a third route, extracting every provable English dialogue clip straight to a WAV file up front.

The catalog columns are `line_id`, `core_path`, `line_index`, `category`, `scene`, `speaker_code`, `speaker_name`, `subtitle_en`, `wem_path_en` and `language`. There is no per-line length or duration column. How long a clip runs is not known here; it is measured later, by decoding the clip during render. The GUI's library view shows a length column, but it fills that by decoding each clip on a background thread, not from the catalog.

Resume is deliberate. A line's row counts as finished only once a separate sidecar file confirms its whole source core completed, and a startup pass drops any rows left behind by a source that crashed partway. That is what stops a half-processed file from looking done and quietly losing the rest of its lines on the next run. When the Horizon path-scan cannot read a core, it no longer drops it silently: the failure is counted and written to an errors log, the same as every neighbouring stage.

IN	the game install
DOES	enumerates dialogue lines and their metadata into a CSV
OUT	<code>catalog.csv</code> (DS, HZD) or <code>clip-index.csv</code> (FW), plus a resume sidecar
NEEDS	the Oodle DLL (DS); nothing external for HZD's scan; VGAudio for FW's decode
FILES	<code>games/ds/catalog.py</code> <code>games/hzd/catalog.py</code> <code>games/hzd/inventory.py</code> <code>games/fw/extract.py</code>

Bind: attach a clip to each line

This is where the three games stop resembling each other, and it is the reason the project is really three solutions sharing a frame. Death Stranding has no bind step at all: the catalog already names each line's audio, so there is nothing to match, and cutscene voice comes from whole-scene tracks resolved once and shipped with the tool.

Horizon Zero Dawn starts with lines and clips that are both anonymous. Each is fingerprinted by two cheap numbers, its encoded byte length and its sample count. A fingerprint that matches exactly one line to exactly one clip is bound immediately with no further work, marked tier S. Fingerprints shared by several lines and clips are ambiguous, and only those are handed to a local speech-to-text model and matched by their text.

Forbidden West transcribes every clip with the same local model, assigns the game's exact on-screen subtitle to each clip (using the transcript only to pick which clip within a group), then matches each subtitled line against your own copy of the game's script. The script is the story spine: a line with no home in the script is a bark and never binds, and the script also supplies the speaker and the running order. The Horizon binds can run for hours on a full library.

The `tier` field records how a row was bound, not what kind of line it is. Tier S is a fingerprint bind; other tiers mark subtitle, woven and DLC binds. Render later selects rows by tier. It is not a bark-versus-dialogue flag, and there is no separate field that is. The Horizon bind now writes a coverage summary to `coverage.json` beside the manifest, recording how many ambiguous groups its cap left untranscribed, so a partial rip is distinguishable from a complete one on disk, not only from a line in the log.

IN	the catalog plus the game's raw audio clips
DOES	decides which recorded clip carries which line
OUT	a bind manifest, with a <code>tier</code> on each row recording how it was matched
NEEDS	Death Stranding, nothing; Horizon games, a CUDA GPU and the <code>asr</code> extra
FILES	<code>games/hzd/asr_bind.py</code> <code>games/fw/subtitle_bind.py</code> <code>games/fw/subtitle_match.py</code> <code>engine/coverage.py</code>

Order: put the lines in story sequence

Death Stranding has a real, separate order step. It keeps only lines that have a subtitle, removes within-scene duplicates, and places scenes in narrative order, anchored to a transcript you supply where one covers a scene and falling back to built-in heuristics elsewhere. The result is `playlist.csv`, tagged so that cutscene and mission lines form the spine and everything else is side content.

The Horizon games have no standalone order command. Their order is a byproduct of the bind: Horizon Zero Dawn uses quest numbering, Forbidden West uses the position of each line in your script. That order is already written into the bind manifest, so there is no separate marker or file for it.

IN	the catalog (DS) or the bind manifest (Horizon)
DOES	selects and orders lines into a playable sequence
OUT	<code>playlist.csv</code> (DS); order baked into the manifest (Horizon)
NEEDS	an optional narrative transcript (DS) for tighter placement
FILES	<code>games/ds/story_order.py</code> <code>engine/selection.py</code> <code>engine/transcript_anchor.py</code>

Render: pack into MP3 reels

Render decodes each clip to WAV, caching as it goes so re-runs are cheap, drops a short silence between lines and a longer one between scenes, and concatenates the result into MP3 files. It sizes each file to land just under 290 MB. Two levers control that now: `--target-mb` sets the size each reel aims for (285 MB by default), and `--bitrate` sets the audio quality, with lower bitrates fitting more audio into each reel. Each reel gets a tracklist sidecar carrying the timestamp, scene or quest, speaker, subtitle and line id.

Rendering a chosen subset of lines is a first-class feature. Render reads its lines from a playlist or manifest you can point elsewhere, and filters rows by tier (Forbidden West exposes this as `--tiers`, Death Stranding has `--main-story`). The GUI's library view makes this direct: tick the lines you want, and the Export panel writes a filtered manifest and renders exactly those. A separate dump stage, and the GUI's "Dump WAV" button, decode the ticked lines to individual WAV files instead of packing reels.

Render will not fake success. If it attempted to decode clips and none of them worked, it stops with an error that points at `doctor` and the per-clip error log, rather than writing a silent, empty reel. An empty input is likewise a loud stop, not a quiet do-nothing.

IN	the ordered playlist or manifest
DOES	decodes each clip and concatenates the clips into sized MP3 files
OUT	<code>phase_d_NN.mp3</code> plus <code>phase_d_NN.tracklist.csv</code> per reel
NEEDS	ffmpeg, plus the game's own decoder (vgmstream and Oodle, or VGAudio)
FILES	<code>engine/render.py</code> <code>games/hzd/render.py</code> <code>games/fw/render.py</code> <code>gui/export_model.py</code>

4.

The map

<code>src/deciwaves/gui/</code>	The desktop app: a thin PySide6 front-end that runs deciwaves commands and shows their progress. The decision logic sits in Qt-free <code>*_model.py</code> files; the widgets just draw.
<code>src/deciwaves/engine/</code>	The shared, game-agnostic core: the render, selection, coverage and resume-bookkeeping code every game reuses, plus decoder-path resolution.
<code>src/deciwaves/engine/pack/</code>	The format readers, one family per game: DS archives, the HZD package, the FW streaming graph.
<code>src/deciwaves/games/ds hzd fw/</code>	Each game's own catalog and bind code, and Death Stranding's own order step. This is where the three approaches actually live.
<code>src/deciwaves/cli/</code>	The commands: main dispatches (and launches the GUI), run chains a game end to end, setup and doctor handle readiness, guided is the fallback menu.
<code>src/deciwaves/_vendor/pydecima/</code>	A bundled, patched Decima resource parser, used by Death Stranding only.
<code>src/deciwaves/data/</code>	Small reference files shipped with the tool: file lists, pre-resolved cutscene tracks, character rosters.
<code>out/</code> (per workspace)	Everything a run produces: the CSVs, the WAV cache, the MP3 reels, the coverage summaries, and the <code>.done-<stage></code> resume markers. Kept out of git.
<code>.memories/</code>	Durable notes on the reverse-engineered formats. The best place to understand why a game needed the solution it got.

5.

How to run it

1. Install the package

```
pip install -e ".[gui]" # the desktop app; add [asr] as well for the
Horizon binds
```

The `[gui]` extra pulls in PySide6. Base install (no `[gui]`) is CLI-only. The Horizon binds need the `asr` extra plus a CUDA-matched PyTorch. `[test]` adds pytest.

2. Open the GUI, or use the CLI

```
deciwaves # opens the desktop app (or falls back to
a guided prompt)
deciwaves setup --ds-install "D:\ ... \DEATH STRANDING DIRECTOR'S CUT"
deciwaves doctor
```

In the GUI, Setup and Doctor are panels. HZD wants the ...\\LocalCacheDX12\\package dir (with PackFileLocators.bin); FW wants ...\\LocalCacheWinGame\\package (with streaming_graph.core).

3. Run a game end to end

```
deciwaves --workspace D:\\deciwaves ds run
deciwaves --workspace D:\\deciwaves hzd run
deciwaves --workspace D:\\deciwaves fw run --gamescript <your-script>
```

The --workspace flag MUST come before the game name. Forbidden West produces subtitle-labelled reels without a script, but needs your own script for speakers and story order. In the GUI this is the Scan then Process buttons.

4. Collect the output

```
dir D:\\deciwaves\\out\\ds
```

The manifest CSV and the phase_d_NN.mp3 reels with their tracklists land under out\\<game>.

6.

What could bite you

The classic pipeline hazards are handled here, and it is worth saying so plainly. Files are written to a temporary name and moved into place, so a crash never leaves a half-written file looking valid. Resume state has one authority, so a re-run cannot silently lose work. Re-running an early stage automatically invalidates the later stages' markers, so stale downstream output cannot pass for fresh. Two problems the previous report flagged are now fixed: the Horizon path-scan logs the cores it cannot read, and a capped bind writes its skipped-group count to disk. What remains is smaller, and mostly about the gap between starting a job and its finishing in reasonable time.

On the command line, a GPU stage will start on a machine with no GPU and run for days

MEDIUM

FIX: M

src/deciwaves/cli/run.py, lines 151, 179-182

RISK Before a Horizon bind, the command-line tool checks only whether the speech-to-text software is installed, not whether a graphics card is actually present. A machine with the software but a CPU-only setup passes the check, the stage starts, and it then runs for hours or days on the processor with no warning. The desktop GUI closes this gap: it asks **doctor** whether a card is visible and pops a confirmation first. Anyone driving **deciwaves hzd run** or **deciwaves fw run** straight from a terminal gets no such prompt.

FIX Have the command-line GPU gate check for a usable CUDA device, not just that the package imports, and refuse or warn when none is found. The GUI already has the check that is needed; the same test belongs in the shared gate.

Two smaller loose ends round it out, both low and neither able to corrupt output. The `.done-bind` marker does not record the cap the run used, so re-running a Horizon bind with a larger `--sample-cap` is silently skipped as already done; force the fuller pass with `deciwaves hzd run --from bind`, which deletes the marker for you. And `--workspace` must come before the game name: placed after it, it is swallowed as a stage argument and the run stops with a misdirected “invalid choice: ‘-workspace’”. GUI users hit neither.

7.

Open questions

? Forbidden West ships three ways to assemble a reel: full, woven, and a separate DLC path. Both `fw run` and the GUI use only the full reel. Whether the other two are finished features or exploratory leftovers is not decidable from the code alone.

? The Death Stranding `cutscenes` stage is not in the default chain, which uses a pre-resolved cutscene-track file shipped with the tool. Whether users should regenerate it against their own install, or treat the shipped file as canonical, is a product decision the code does not settle.

8.

Glossary

Decima The game engine all three games are built on. DeciWaves reads its data files; it does not run the engine.

sentence core A Decima data record describing one line of dialogue: its text, speaker, and a reference to its audio.

Wwise / .wem The audio format the recorded voice sits in inside Death Stranding. Decoding it needs a Wwise-aware tool, `vgmstream`.

ATRAC9 The compressed audio format the two Horizon games use for their clips, decoded with `VGAudio`.

Oodle A compression library Death Stranding uses. Its DLL ships next to the game’s executable, and DeciWaves needs it to open the archives.

fingerprint Two cheap numbers, a clip’s encoded byte length and its sample count, used to tell clips apart without decoding them. A match made from these alone, no speech-to-text, is marked tier S.

speech-to-text (ASR) Turning recorded speech into text, run locally on the GPU. Used only to disambiguate clips a fingerprint cannot separate.

gamescript The game’s own script, in play order. A file you supply yourself; it gives Forbidden West its speakers and story order.

tier A tag on each bound line recording how it was matched. Render selects which tiers to include. Not a line-type label. A short unscripted line with no place in the script, a guard’s shout or idle mutter, is a “bark” and never binds.

coverage summary A `coverage.json` file recording how complete a stage’s output is, e.g. how many groups a capped bind skipped. An empty `.done-<stage>` marker beside it records that a stage finished, so a re-run skips it.